SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

---

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Debugging or Optimization task

Code of Conduct:

I S.MAHALAKSHMI(192524372) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.MAHALAKSHMI

Q1. Deadlock Debugging

1. Problem statement and identification

Process P1 locks R1, waits for 100 ms, and then requests R2. Process P2 locks R2, waits for 100 ms, and then requests R1. If P1 holds R1 while P2 holds R2, both processes wait for a resource held by the other. Neither process can enter its critical section or release its first lock. This is a deadlock.

2. Root cause and deadlock conditions

The root cause is that the processes acquire the same resources in opposite orders. The 100 ms delay increases the chance that each process acquires its first resource before requesting the second; however, the delay is not itself the fundamental cause.

- Mutual exclusion: R1 and R2 can each be locked by only one process at a time.
- Hold and wait: each process holds one resource while waiting for the other.
- No preemption: the system does not forcibly take a locked resource from its owner.
- Circular wait: P1 waits for R2 held by P2, and P2 waits for R1 held by P1.

3. Debugging approach

1. Trace every lock acquisition and release, including the process/thread ID and timestamp.
2. Check whether P1 owns R1 while waiting for R2 and P2 owns R2 while waiting for R1.
3. Draw the wait-for graph: P1 → P2 and P2 → P1. The cycle confirms circular wait.
4. Use a timeout or deadlock detector during testing so a test does not hang indefinitely.

4. Optimization

Use a single global lock order. Both processes must acquire R1 first and R2 second, then release the locks in reverse order. This breaks the circular-wait condition without changing the critical-section operation.

5. Corrected Python demonstration

```
import threading
import time

R1 = threading.Lock()
R2 = threading.Lock()

def process(name):
    print(name, "requesting R1")
    with R1:                    # Always acquire R1 first
        print(name, "acquired R1")
        time.sleep(0.05)
        print(name, "requesting R2")
        with R2:                # Then acquire R2
            print(name, "acquired R2")
            print(name, "entered critical section")
            time.sleep(0.02)
        print(name, "released R2")
    print(name, "released R1")

t1 = threading.Thread(target=process, args=("P1",))
t2 = threading.Thread(target=process, args=("P2",))
t1.start()
t2.start()
```

```
t1.join()
t2.join()
print("DEBUG: both processes completed; no deadlock")
```

Note: Python's thread output order may vary, but both threads should finish because they use the same lock order.

Expected debug output:

```
P1 requesting R1
P1 acquired R1
P2 requesting R1
P1 requesting R2
P1 acquired R2
P1 entered critical section
P1 released R2
P1 released R1
P2 acquired R1
P2 requesting R2
P2 acquired R2
P2 entered critical section
P2 released R2
P2 released R1
DEBUG: both processes completed; no deadlock
```

6. Analysis and conclusion

Consistent resource ordering prevents a cycle in the wait-for graph. The test output confirms that each thread eventually enters and exits the critical section. In production code, lock timeouts and diagnostic logging can further help detect unexpected stalls.

Q2. CPU Utilisation Problem

1. Problem identification

The system reports only 30% CPU utilization even though processes are ready. The observed ready → waiting → ready → waiting sequence suggests frequent blocking and wakeups, often due to I/O requests, locks, or synchronisation events. CPU utilization can remain low when processes spend much of their time waiting for I/O or when the ready queue temporarily becomes empty.

2. Root cause analysis

The state sequence alone does not prove one exact cause. The debugging task is to identify what each process is waiting for. A process in the waiting/blocked state is not eligible to use the CPU until the event completes; changing CPU priority cannot directly remove an I/O delay.

3. Debugging procedure

5. Record timestamps for running, ready, and blocked/waiting state transitions.
6. Record the wait reason: disk, network, keyboard, mutex/semaphore, timer, or another event.
7. Check CPU usage, ready-queue length, I/O latency, lock contention, and voluntary context switches.
8. Determine whether the CPU is idle because all tasks are blocked, or whether runnable tasks are delayed by scheduling overhead.

4. System-level optimization

- Use asynchronous or buffered I/O where appropriate so one slow I/O operation does not block unrelated work.
- Batch many small I/O operations into fewer larger operations when the application permits it.
- Replace busy-wait loops with blocking locks, condition variables, or event notifications.

- Use a bounded worker pool or event-driven architecture to overlap independent tasks without creating excessive threads.
- Tune scheduler parameters only after collecting measurements; avoid assuming a scheduler change will fix an I/O bottleneck.

5. Python debug-log example

This small diagnostic example illustrates how state transitions can be logged. It does not reproduce a real operating-system scheduler; it shows the kind of evidence to collect.

```
events = [
    ("P1", "READY", "task submitted"),
    ("P1", "RUNNING", "dispatched by scheduler"),
    ("P1", "WAITING", "disk read requested"),
    ("P2", "READY", "task submitted"),
    ("P2", "RUNNING", "dispatched by scheduler"),
    ("P2", "WAITING", "network response requested"),
    ("P1", "READY", "disk read completed"),
    ("P2", "READY", "network response received"),
]

for pid, state, reason in events:
    print(f"DEBUG: {pid:>2} -> {state:<7} | {reason}")

waiting = sum(1 for _, state, _ in events if state == "WAITING")
print("DEBUG: waiting-state transitions =", waiting)
print("ACTION: inspect I/O latency and blocked time before tuning scheduler")
```

Expected debug output:

```
DEBUG: P1 -> READY   | task submitted
```

```
DEBUG: P1 -> RUNNING | dispatched by scheduler
DEBUG: P1 -> WAITING | disk read requested
DEBUG: P2 -> READY   | task submitted
DEBUG: P2 -> RUNNING | dispatched by scheduler
DEBUG: P2 -> WAITING | network response requested
DEBUG: P1 -> READY   | disk read completed
DEBUG: P2 -> READY   | network response received
DEBUG: waiting-state transitions = 2
ACTION: inspect I/O latency and blocked time before tuning scheduler
```

6. Expected outcome and conclusion

If blocking I/O is the cause, asynchronous I/O, batching, and reduced lock contention can let other work proceed while one operation waits. Validate the change by comparing CPU utilization, blocked time, throughput, response time, and I/O latency before and after optimization.

Q3. Memory Bottleneck Debugging

1. Problem identification and calculation

The program reports frequent page faults. Given values: code = 5 MB, heap = 150 MB, stack = 1 MB per thread, and 25 threads. The nominal total of these listed areas is 5 + 150 + (25 × 1) = approximately 180 MB. This estimate excludes shared libraries, other allocations, page tables, memory used by other processes, and operating-system overhead.

Because the stated 180 MB is below 2 GB of physical RAM, the supplied numbers alone do not prove that the machine has run out of RAM. Page faults can also occur because of poor locality, first-time page access, memory pressure caused by other processes, or a working set larger than the available resident memory.

2. Root cause possibilities

- Working-set pressure: the program repeatedly accesses more pages than can stay resident.
- Poor locality: code or data access jumps across a large memory region, causing pages to be loaded repeatedly.
- Excessive allocation: temporary objects or duplicated data enlarge the heap.
- Thread overhead: 25 threads reserve stack space and add scheduling/synchronisation overhead; actual stack commitment depends on the platform and use.

3. Debugging procedure

9. Measure major and minor page faults separately.
10. Monitor resident-set size, available RAM, swap usage, and memory usage of other processes.
11. Profile allocations to find large objects, duplicates, and objects that remain alive unnecessarily.
12. Inspect data-access patterns and determine whether the active working set fits in available memory.
13. Measure per-thread stack use before reducing stack sizes.

4. Optimization 1 — reduce the working set and improve locality

Release unused objects, avoid unnecessary copies, process large files or datasets in chunks, and access arrays sequentially where possible. These changes reduce the number of pages that must remain active and may lower expensive major page faults.

5. Optimization 2 — right-size threads and stacks

Use a bounded thread pool when the workload does not need 25 simultaneous threads. Configure thread-stack sizes only after measuring real stack usage and checking the platform's requirements. Do not set stack sizes too low, because deep recursion or large local variables can cause stack overflow.

6. Illustrative Python page-fault/working-set experiment

The following program simulates page references and counts page misses under a simple LRU replacement policy. It is an educational simulation, not a measurement of actual operating-system page faults.

```
def lru_page_faults(reference_string, frame_count):
    frames = []
    faults = 0
    for page in reference_string:
        if page in frames:
            frames.remove(page)       # Recently used page becomes newest
            frames.append(page)
            print(f"HIT  page={page} frames={frames}")
        else:
            faults += 1
            if len(frames) == frame_count:
                removed = frames.pop(0)
                print(f"FAULT page={page}; evict={removed}")
            else:
                print(f"FAULT page={page}; free frame")
            frames.append(page)
    return faults


refs = [1, 2, 3, 1, 4, 2, 5, 1]
faults = lru_page_faults(refs, frame_count=3)
print("Total simulated page faults:", faults)
```

Expected debug output:

```
FAULT page=1; free frame
```

```
FAULT page=2; free frame
FAULT page=3; free frame
HIT  page=1 frames=[2, 3, 1]
FAULT page=4; evict=2
FAULT page=2; evict=3
FAULT page=5; evict=1
FAULT page=1; evict=4
Total simulated page faults: 7
```

7. Analysis and conclusion

In the example, only three simulated page frames are available, so page references that are no longer resident cause misses. Improving locality or increasing the effective available memory for the working set can reduce faults. For the real application, use operating-system profiling tools to confirm the cause before changing memory or thread settings.

Q4. Starvation Debugging

1. Problem identification

P5 is assigned to Q3, the lowest-priority batch queue. Q1 is the highest-priority interactive queue with a 5 ms quantum, Q2 is the system queue with a 10 ms quantum, and Q3 uses FCFS. If the scheduler always serves Q1 and Q2 whenever they have work, a continuously busy higher-priority queue can postpone P5 indefinitely. This is starvation.

2. Root cause

The root cause is strict priority between queues and the absence of a stated fairness mechanism. FCFS within Q3 only determines the order among processes in Q3; it does not guarantee that Q3 will receive CPU time.

3. Debugging procedure

14. Log the time each process enters a queue and the time it first receives CPU time.
15. Calculate waiting time for P5 and other Q3 processes.
16. Check whether Q1 or Q2 remains continuously non-empty.
17. Confirm whether the scheduler includes aging, priority boosts, or a guaranteed CPU share for lower queues.

4. Optimization for fairness

Apply aging: increase the priority of a process the longer it waits, or promote it to a higher queue after a configured waiting threshold. A second option is to reserve a minimum percentage of CPU time for Q3. Combining aging with a guaranteed CPU share provides both eventual service and a predictable opportunity for batch work.

5. Python aging demonstration

This compact model illustrates the principle: a process waiting longer receives a higher effective priority. Lower numeric values represent higher priority in this example.

```
processes = [
    {"name": "P1", "base": 1, "wait": 0},
    {"name": "P2", "base": 2, "wait": 2},
    {"name": "P5", "base": 3, "wait": 8},
]

AGING_INTERVAL = 3
for p in processes:
    boosts = p["wait"] // AGING_INTERVAL
    p["effective"] = max(0, p["base"] - boosts)
    print(f"DEBUG: {p['name']} base={p['base']} wait={p['wait']}ms "
          f"boosts={boosts} effective_priority={p['effective']}")
```

```
selected = min(processes, key=lambda p: (p["effective"], -p["wait"]))
print("Scheduler selects:", selected["name"])
```

Expected debug output:

```
DEBUG: P1 base=1 wait=0ms boosts=0 effective_priority=1
DEBUG: P2 base=2 wait=2ms boosts=0 effective_priority=2
DEBUG: P5 base=3 wait=8ms boosts=2 effective_priority=1
Scheduler selects: P5
```

This is a simplified illustration, not a complete multilevel queue scheduler. A production implementation must define queue promotion/demotion rules and prevent priority inversion or unfairness elsewhere.

6. Analysis and conclusion

Aging prevents a process from remaining at low priority forever, provided the scheduler eventually dispatches promoted processes. A reserved CPU share for Q3 can also provide a measurable fairness guarantee. Validate the fix by checking that maximum waiting time decreases and every continuously ready batch process eventually runs.

Q5. Context-Switch Overhead Debugging

1. Problem identification

The server performs 20,000 context switches per second. The average thread burst is 2 ms, I/O waits are very frequent, and the preemptive scheduler uses a 1 ms time quantum. A short quantum can cause a CPU-bound thread to be preempted before completing its burst. Frequent I/O blocking and wakeups also cause threads to leave and re-enter the runnable queue.

2. Root cause and evidence to collect

- The 1 ms quantum is short relative to the 2 ms average burst, potentially increasing timer-driven preemption.
- Frequent I/O waits can create many block/wakeup transitions and scheduling decisions.
- A large number of runnable threads or lock contention can increase scheduling activity.
- The given details do not prove that the quantum alone accounts for all 20,000 switches/sec; collect voluntary/involuntary switch counts, run-queue length, I/O latency, and CPU time spent in the kernel.

3. Optimization 1 — benchmark a larger quantum

Test a moderate increase, such as 2–4 ms, and compare context switches per second, throughput, response time, and fairness. A larger quantum may reduce preemption but can make interactive requests less responsive. Choose the value from measurements rather than assuming one value is universally best.

4. Optimization 2 — reduce avoidable blocking and wakeups

Use asynchronous or batched I/O where suitable, reduce unnecessary lock contention, avoid waking threads that cannot make progress, and use a bounded worker pool if too many threads are competing. These changes reduce unnecessary block/wakeup cycles and scheduling overhead.

5. Python Round Robin simulation

This simulation compares context switches between different time quanta for three CPU bursts. It counts a switch when the CPU moves from one process to a different process; it excludes the initial dispatch and final switch to idle. Real operating-system counts also include blocking, wakeups, interrupts, and other events.

```
from collections import deque
```

```
def round_robin_switches(bursts, quantum):
    ready = deque([[name, burst] for name, burst in bursts.items()])
    previous = None
    switches = 0
    timeline = []
    while ready:
        name, remaining = ready.popleft()
        if previous is not None and previous != name:
            switches += 1
        run_for = min(quantum, remaining)
        timeline.append(f"{name}:{run_for}ms")
        remaining -= run_for
        previous = name
        if remaining > 0:
            ready.append([name, remaining])
    return switches, timeline

bursts = {"P1": 5, "P2": 4, "P3": 3}
for q in (1, 2, 3):
    switches, timeline = round_robin_switches(bursts, q)
    print(f"quantum={q}ms switches={switches}")
    print("timeline:", " | ".join(timeline))
```

Expected debug output:

```
quantum=1ms switches=11
timeline: P1:1ms | P2:1ms | P3:1ms | P1:1ms | P2:1ms | P3:1ms | P1:1ms | P2:1ms |
P3:1ms | P1:1ms | P2:1ms | P1:1ms
quantum=2ms switches=5
```

```
timeline: P1:2ms | P2:2ms | P3:2ms | P1:2ms | P2:2ms | P3:1ms | P1:1ms
quantum=3ms switches=4
timeline: P1:3ms | P2:3ms | P3:3ms | P1:2ms | P2:1ms
```

6. Analysis and conclusion

In this simple simulation, larger quanta reduce process-to-process switches because each dispatch performs more work before yielding. However, real workloads include I/O blocking and interactive latency requirements, so the best quantum must be measured. Combine scheduler tuning with reduced unnecessary I/O waits and wakeups, then compare performance counters before and after the change.

Overall Conclusion

The five cases show that effective operating-system optimization begins with evidence-based debugging. Deadlock is prevented by consistent resource ordering; low CPU utilization requires investigation of blocked processes and I/O; page-fault problems require working-set and memory profiling; starvation requires aging or guaranteed CPU allocation; and excessive context switching can be reduced through measured scheduler tuning and fewer avoidable wakeups. In every case, validate the proposed fix with logs and repeatable measurements.